



UNIVERSIDAD
DE GRANADA

Universidad de Granada

Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

GRADO EN INGENIERÍA INFORMÁTICA

DIRECCIÓN Y GESTIÓN DE PROYECTOS (DGP)

METODOLOGÍAS DE DESARROLLO ÁGIL (MDA)

Curso Académico 2026–2027

MANUAL DE COORDINACIÓN Y PLAN DE DIRECCIÓN DEL PROYECTO

Proyecto AUTOVIDA: Sistema de Agenda y Asignación de Tareas Accesible para Alumnado NEAE / PTVL

En colaboración con el C.E.E. Fundación Purísima Concepción de Granada

Equipo de Trabajo:

**CLENCH Software
Development**



Asignación de Roles:

Miguel Ángel Luque

Yeray Rodríguez Navas

Julián Carrión Tovar

Pablo Hernández Ibáñez

Pablo de la Torre Roldán

José Rodríguez Fernández

Coordinador

Gestor de Calidad

Gestor Accesibilidad

Moderador

Catalogador

Presentador

Versión 2.0 — Octubre de 2026

Firma y Aceptación de los Miembros del Equipo

El presente documento constituye el marco formal de organización, coordinación y dirección técnica del equipo de trabajo constituido para las asignaturas de **Dirección y Gestión de Proyectos (DGP)** y **Metodologías de Desarrollo Ágil (MDA)** del Grado en Ingeniería Informática de la Universidad de Granada durante el curso académico 2026–2027.

Todos los miembros firmantes han participado en su redacción, consienten las normas de convivencia, roles asignados, mecanismos de seguimiento y asumen la responsabilidad compartida e individual de los compromisos adquiridos.

Nombre y Apellidos	Rol Principal (Fase Inicial)	Asignatura(s)	Firma y Conformidad
Miguel Ángel Luque	Coordinador (Rotativo)	DGP / MDA	<i>Firmado electrónicamente</i>
Yeray Rodríguez Navas	Gestor de Calidad (Fijo)	DGP / MDA	<i>Firmado electrónicamente</i>
Julián Carrión Tovar	Gestor Accesibilidad (Fijo)	DGP / MDA	<i>Firmado electrónicamente</i>
Pablo Hernández Ibáñez	Moderador (Rotativo)	DGP / MDA	<i>Firmado electrónicamente</i>
Pablo de la Torre Roldán	Catalogador (Rotativo)	DGP / MDA	<i>Firmado electrónicamente</i>
José Rodríguez Fernández	Presentador (Rotativo)	DGP / MDA	<i>Firmado electrónicamente</i>

Propósito y Validez del Documento

Este manual regula el ciclo de vida del desarrollo ágil, el reparto equitativo de roles y cargas, la custodia y estándares de documentación textual y código, la relación semanal con el cliente, el flujo de gestión de cambios, los protocolos de calidad y el régimen disciplinario y de recompensas conforme a las directrices de DGP y MDA.

Índice

Firma y Aceptación de los Miembros del Equipo	i
1. Imagen de Empresa	1
1.1. Identidad Corporativa: CLENCH Software Development	1
2. Metodología de Desarrollo y Ciclo de Vida	1
2.1. Modelo Metodológico Ágil	1
2.2. Ciclo de Vida por Iteraciones	1
2.3. Eventos y Ceremonias Ágiles Adaptadas	2
3. Recursos Software de Desarrollo	2
3.1. Ecosistema y Herramientas de Desarrollo	2
4. Organización del Equipo de Trabajo (Estructura, Normas)	3
4.1. Asignación Nominal de Roles Iniciales	3
4.2. Responsabilidades de Cada Rol	3
4.3. Fijeza, Esquema de Rotación y Protocolo de Ausencias	5
4.4. Matriz de Rotación de Roles por Iteración	5
4.5. Normas de Convivencia y Compromiso Operativo	5
5. Herramientas para Comunicaciones en el Equipo y Reuniones	5
5.1. Canales Oficiales del Equipo	5
5.2. Protocolo de Reuniones y Elaboración de Actas	6
6. Relaciones con el Cliente (Entrevistas, Reuniones, Revisiones)	6
6.1. Dinámica de Colaboración Semanal y Entrevistas	6
6.2. Interlocutores y Ética en el Trato	6
7. Estándares de Documentación	6
7.1. Documentación Textual en LaTeX a través de GitHub y Estructura Normalizada . .	6
7.2. Documentación de Código: Doxygen	7
7.3. Diseño y Modelado Software: UML con Visual Paradigm	7
8. Estándares de Código	7
8.1. Convenciones de Nomenclatura y Lenguaje	8
8.2. Comentarios y Documentación en Código	8
8.3. Principios de Calidad y Formato	8
9. Plan de Gestión de Cambios	8
9.1. Procedimiento de Evaluación y Aprobación	8
10. Control de Versiones (Método y Herramientas)	8
10.1. Estrategia y Disciplina en el Repositorio	9
10.2. Convención de Commits (Conventional Commits)	9
11. Gestión de Calidad (Durante el Desarrollo y al Final, Herramientas)	9
11.1. Procedimiento de Calidad en Cada Iteración	9
11.2. Proceso de Hardening y Verificación Global	10
11.3. Cuadro Oficial de Herramientas de Calidad	10

12. Plan de Medición del Desempeño, Recompensas y Castigos	10
12.1. Registro y Control del Esfuerzo Individual	10
12.2. Sistema de Recompensas y Reconocimiento	10
12.3. Régimen Sancionador Interno (Castigos y Penalizaciones)	11

1. Imagen de Empresa

1.1. Identidad Corporativa: CLENCH Software Development

El equipo de trabajo opera bajo la identidad corporativa y marca registrada **CLENCH Software Development**. La iniciativa surge con un marcado carácter de **Aprendizaje-Servicio**, aunando la excelencia en ingeniería de software con la responsabilidad social y la inclusión de colectivos vulnerables.



- **Denominación Oficial:** CLENCH Software Development.
- **Lema y Propósito:** «Tecnología accesible para transformar vidas y potenciar la autonomía personal».
- **Misión:** Diseñar, implementar y validar soluciones digitales accesibles, robustas e intuitivas que eliminen barreras cognitivas y motrices en la rutina diaria de estudiantes con Necesidades Específicas de Apoyo Educativo (NEAE).
- **Visión:** Consolidar un modelo de equipo multidisciplinar altamente cohesionado, referente en buenas prácticas de metodologías ágiles y cumplimiento estricto de las directrices universales de accesibilidad (WCAG 2.1 AA).
- **Valores Corporativos:**
 1. **Accesibilidad Universal y Empatía:** La persona usuaria (estudiantes PTVL y profesorado de la Fundación Purísima Concepción) es el núcleo de cada decisión de diseño.
 2. **Rigor Técnico y Calidad Continua:** Compromiso innegociable con el código limpio, análisis estático, cobertura de pruebas y estabilidad.
 3. **Transparencia y Coordinación:** Comunicación fluida, reparto equitativo de esfuerzo y sinceridad en la estimación de plazos y riesgos.
 4. **Ética y Protección de Datos:** Respeto absoluto a la privacidad, dignidad y normativa RGPD en el tratamiento de información asistencial.

2. Metodología de Desarrollo y Ciclo de Vida

2.1. Modelo Metodológico Ágil

Para desarrollar la aplicación utilizaremos ****metodología ágil****, integrando los eventos de trabajo de Scrum con tableros Kanban para la gestión del flujo de trabajo continuo y la monitorización de tareas en curso (*Work In Progress*). Este esquema equilibra la flexibilidad en la asimilación de feedback de usuarios reales con el control formal de alcance, costes, riesgos y plazos de DGP.

2.2. Ciclo de Vida por Iteraciones

El proyecto se ejecuta bajo un modelo de **prototipado evolutivo iterativo**, articulado en cuatro hitos principales y una fase final de verificación global:

1. **Fase Inicial (Entrega 21–22 de octubre de 2026):** Constitución formal del equipo, firma del Manual de Coordinación, propuesta técnica preliminar con arquitectura y estimación de costes, definición de Personajes y Escenarios (Po y P1), y consolidación del Product Backlog inicial y Plan de Entregas (P2).
2. **Iteración 1 (22 de octubre – 11/12 de noviembre de 2026):** Puesta a punto del entorno en Flutter, arquitectura base, autenticación adaptada y módulo de perfiles de usuario accesibles. Primer prototipo funcional evaluable.
3. **Iteración 2 (12 de noviembre – 2/3 de diciembre de 2026):** Implementación de tareas escolares (flujo guiado de reprografía, gestión de comedores y dietas especiales, y guía de visitas) e incorporación del temporizador visual accesible.
4. **Iteración 3 (3 de diciembre – 21 de diciembre de 2026):** Módulo de tareas personales configurables, chat accesible tutor–estudiante, refinamiento de contrastes y áreas táctiles, y estabilización integral del sistema.
5. **Hardening y Verificación Global (Cierre de Iteración 3):** Fase de endurecimiento y verificación exhaustiva que incluye revisión con el cliente, pruebas extremas y auditoría final de políticas de datos (RGPD) y vulnerabilidades.
6. **Defensa Final (12 de enero de 2027):** Presentación oral, defensa técnica y demostración interactiva en directo ante el tribunal evaluador y representantes de la institución cliente.

2.3. Eventos y Ceremonias Ágiles Adaptadas

- **Sprint Planning:** Al inicio de cada iteración, el equipo selecciona historias de usuario del backlog, estima su dedicación en horas reales y las desglosa en tareas técnicas en ClickUp.
- **Sincronización Semanal (Weekly Standup):** Reunión presencial obligatoria semanal en la sesión de prácticas con el profesor/tutor, complementada con seguimiento asíncrono en Discord (qué se hizo, qué se hará hoy y qué impedimentos existen).
- **Sprint Review y Demostración:** Exposición del incremento de software funcional desplegable ante el cliente y profesorado para validar los requisitos implementados.
- **Sprint Retrospective:** Evaluación interna del rendimiento grupal, precisión de estimaciones y definición de medidas de mejora operativa para el siguiente ciclo.

3. Recursos Software de Desarrollo

La aplicación se desarrollará principalmente en **Flutter**. Trabajaremos con el **IDE de Flutter** por las facilidades que nos brinda a la hora de hacer tests unitarios e interfaces personalizables en tiempo de ejecución.

3.1. Ecosistema y Herramientas de Desarrollo

- **Framework de Desarrollo Frontend: Flutter SDK (Dart).** Permite generar aplicaciones nativas multiplataforma desde un único código fuente, asegurando un rendimiento fluido en las tabletas Android del colegio y compatibilidad web para pizarras digitales.
- **Entorno de Desarrollo (IDE de Flutter): Visual Studio Code / Android Studio** con las extensiones oficiales de Flutter y Dart. Ofrece:

- *Hot Reload* y *Hot Restart* para modificar y personalizar componentes gráficos y estilos visuales en tiempo de ejecución de manera instantánea.
 - Suite integrada para ejecución automatizada y depuración de pruebas unitarias (*Flutter Test*).
 - *Flutter DevTools* para análisis de rendimiento, inspección del árbol de widgets y perfiles de memoria.
- **Backend y API REST: Node.js con Express y TypeScript**, aportando tipado seguro, arquitectura desacoplada y alta escalabilidad en la gestión de servicios web.
 - **Persistencia y Base de Datos: PostgreSQL** gestionado a través de **Prisma ORM**, empaquetado en contenedores Docker para reproducibilidad total de los entornos de ejecución.
 - **Servicios en Tiempo Real: WebSockets / Socket.io** para el intercambio instantáneo de mensajes en el chat tutor–estudiante y la sincronización de estados de tareas.
 - **Catálogo de Apoyos Visuales:** Catálogo oficial de pictogramas de **ARASAAC** (licencia Creative Commons) para comunicación aumentativa.
 - **Diseño UX/UI: Figma** para la confección y testeo de wireframes accesibles previos a la fase de programación.

4. Organización del Equipo de Trabajo (Estructura, Normas)

4.1. Asignación Nominal de Roles Iniciales

Se seguirá una división de roles entre los 6 integrantes del equipo, rotando aquellos roles menos importantes en cada iteración para repartir dicha carga. La asignación inicial de roles del equipo es:

- **Pablo Hernández Ibáñez:** Moderador (Rotativo)
- **Yeray Rodríguez Navas:** Gestor de Calidad (Fijo)
- **Pablo de la Torre Roldán:** Catalogador (Rotativo)
- **José Rodríguez Fernández:** Presentador (Rotativo)
- **Julián Carrión Tovar:** Gestor de Usabilidad y Accesibilidad (Fijo)
- **Miguel Ángel Luque:** Coordinador (Rotativo)

4.2. Responsabilidades de Cada Rol

Cada rol tendrá las siguientes responsabilidades:

- **Coordinador:** Organiza el trabajo del equipo, distribuye tareas, controla los plazos y realiza el seguimiento general del proyecto. Se encarga de avisar sobre las próximas entregas y de planificar el tiempo de trabajo correspondiente a cada iteración. Actúa como portavoz de gestión ante el profesorado y supervisa el equilibrio de la dedicación horaria.
- **Catalogador:** Responsable de recopilar, analizar y clasificar la información generada por el grupo en las distintas tareas asociadas a las prácticas. Se encarga de revisar y consolidar la información añadida a la documentación en cada iteración y, cuando sea posible, de revisar la documentación

de la iteración anterior en coordinación con el Gestor de Calidad. Asimismo, custodia y organiza la documentación en el repositorio institucional (docs/) y coordina la publicación de actas de reunión.

- **Moderador:** Responsable de plantear y moderar los debates, así como de seleccionar las ideas y decisiones grupales. En caso de indecisión, tiene la última palabra tras consultar con el Gestor de Calidad sobre la opción más adecuada. Asimismo, se encarga de gestionar y documentar los cambios en los documentos de especificación.
- **Presentador:** Se encarga de realizar todas las comunicaciones con el cliente, así como de elaborar presentaciones que muestren el avance y los resultados del proyecto. Prepara los resúmenes que se comunicarán al cliente o profesor y decide, junto con los roles pertinentes, qué contenidos y preguntas se expondrán en cada iteración.
- **Gestor de la Calidad (Rol Fijo):** Responsable de asegurar la calidad de los productos generados y del proceso de desarrollo. Vela por que la documentación y las técnicas aplicadas cumplan con los estándares definidos:
 - Asegurarse de que se usan herramientas estandarizadas para especificación y diseño (diagramas UML en Visual Paradigm y plantillas de ingeniería de software).
 - Asegurarse de que se organiza bien el trabajo: celebración regular de reuniones, toma de actas formales, reparto equitativo de tareas y uso de recursos hardware y software correctos.
 - Revisar o asegurarse de que se revisen y prueben todos los entregables (verificando correctitud y completitud) y de que se finalizan a tiempo o se planifica adecuadamente cualquier cambio de fecha y sus consecuencias.
 - Supervisar los criterios de aceptación, revisiones de código, análisis estático continuo y auditorías periódicas de calidad del proceso y producto.
- **Gestor de Usabilidad y Accesibilidad (Rol Fijo):** Responsable de planificar, coordinar y velar por la usabilidad y la accesibilidad del proyecto. Se encarga de investigar e informar al equipo sobre las buenas prácticas, técnicas y herramientas a utilizar en estas materias. Asimismo, define los requisitos de accesibilidad, formula las preguntas necesarias de las consultas que se harán y verifica su correcto cumplimiento mediante pruebas de usuario, testing o entrevistas. Para el desarrollo e implementación de estas actividades contará con el apoyo de los roles correspondientes:
 - Formarse y formar a sus compañeros en guías de usabilidad y accesibilidad web y móvil (WCAG 2.1 AA, heurísticas de accesibilidad cognitiva y diseño accesible).
 - Planificar qué guías considerar en función de los usuarios finales del proyecto (alumnado PTVL con NEAE, docentes y familias).
 - Formarse en heurísticas de accesibilidad y emplearlas rigurosamente para evaluar y probar la aplicación.
 - Aplicar lectores de pantalla (**TalkBack** en Android y **VoiceOver** en iOS) para validar la accesibilidad móvil.
 - Aplicar validadores de accesibilidad (**Lighthouse**, **WAVE**, Google Accessibility Scanner) para evaluar contrastes cromáticos, etiquetas y comprensión.
 - Elaborar informes periódicos del progreso del trabajo en cuanto a usabilidad y accesibilidad.

4.3. Fijeza, Esquema de Rotación y Protocolo de Ausencias

Los roles de **Gestor de Calidad** y **Gestor de Usabilidad y Accesibilidad** serán **fijos** a los responsables correspondientes (Yeray Rodríguez Navas y Julián Carrión Tovar), mientras que el resto rotará en cada iteración de trabajo.

Regla de Rotación Cíclica

La secuencia obligatoria de rotación en cada cambio de iteración es:

... → **Coordinador** → **Catalogador** → **Moderador** → **Presentador** → ...

Protocolo de Suplencias y Ausencias:

1. En caso de que algún integrante con cualquiera de los roles de Gestor (Calidad o Accesibilidad) no pueda acudir a una iteración de trabajo, dicho rol será cedido temporalmente al que en dicha reiteración tenga el rol de **Coordinador** (y al **Presentador** en caso de inasistencia de ambos).
2. Si algún rol rotativo no está presente en una iteración, se le cederá durante esa iteración al responsable de ese rol en la siguiente iteración.

4.4. Matriz de Rotación de Roles por Iteración

Integrante	Fase Inicial	Iteración 1	Iteración 2	Iteración 3
Miguel Ángel Luque	Coordinador	Catalogador	Moderador	Presentador
Pablo de la Torre Roldán	Catalogador	Moderador	Presentador	Coordinador
Pablo Hernández Ibáñez	Moderador	Presentador	Coordinador	Catalogador
José Rodríguez Fernández	Presentador	Coordinador	Catalogador	Moderador
Yeray Rodríguez Navas	Gestor Calidad	Gestor Calidad	Gestor Calidad	Gestor Calidad
Julián Carrión Tovar	Gestor Accesib.	Gestor Accesib.	Gestor Accesib.	Gestor Accesib.

4.5. Normas de Convivencia y Compromiso Operativo

1. **Puntualidad:** Margen máximo de cortesía de 5 minutos en reuniones presenciales y virtuales.
2. **Notificación Previa de Bloqueos:** Notificar obligatoriamente cualquier impedimento o retraso con al menos **48 horas de antelación**.
3. **Registro Honesto de Horas:** Cumplimentación semanal estricta de la ficha de dedicación horaria individual.

5. Herramientas para Comunicaciones en el Equipo y Reuniones

5.1. Canales Oficiales del Equipo

- **WhatsApp:** Principalmente utilizaremos WhatsApp para la organización y concreción de fechas y horarios, así como para la difusión de avisos inmediatos o urgencias imprevistas.
- **Discord:** Plataforma destinada a realizar videollamadas cuando no podamos hacer reuniones presenciales, compartir pantalla en sesiones técnicas y vertebrar discusiones temáticas (#general, #desarrollo-front, #desarrollo-back, #calidad-accesibilidad, #gestion-dgp).

- **ClickUp:** Utilizaremos la herramienta **ClickUp** para el seguimiento y control de las tareas en progreso, gestión del Product Backlog, diagramas de Gantt y órdenes del día de reuniones.
- **Correo Institucional (@correo.ugr.es):** Canal oficial reservado para la comunicación formal con el profesorado y el personal directivo del centro.

5.2. Protocolo de Reuniones y Elaboración de Actas

- **Reuniones Semanales Docentes:** Sesión síncrona obligatoria cada semana en horario de prácticas con el profesor/tutor para revisar el avance del proyecto y validar funcionalidades.
- **Reuniones Semanales Internas:** Encuentro semanal del equipo (presencial o por Discord) convocado formalmente mediante ClickUp.
- **Actas Formales:** Toda reunión oficial se documentará obligatoriamente según la plantilla estandarizada (docs/plantillas/plantilla_acta_reunion.md). El Catalogador y el Moderador redactarán el acta y la publicarán en docs/reuniones/ en un plazo máximo e improrrogable de 24 horas.

6. Relaciones con el Cliente (Entrevistas, Reuniones, Revisiones)

6.1. Dinámica de Colaboración Semanal y Entrevistas

Se realizarán reuniones y revisiones semanales con el cliente/profesor para comprobar la evolución del proyecto, validar las funcionalidades desarrolladas y resolver posibles dudas.

Además, se realizarán **entrevistas específicas** cuando sea necesario para concretar o modificar los requisitos de la aplicación, contrastar la idoneidad pedagógica de los pictogramas o definir flujos de trabajo de las aulas.

Los cambios y decisiones acordados en estas reuniones se registrarán metódicamente para mantener actualizada la planificación y los requisitos del proyecto en todo momento.

6.2. Interlocutores y Ética en el Trato

- **Entorno de Aplicación:** Personal directivo, docentes y profesionales del C.E.E. Fundación Purísima Concepción de Granada.
- **Destinatarios Finales:** Alumnado del Programa de Transición a la Vida Adulta (PTVAL), tutores escolares y familias.
- **Protección de Datos y RGPD:** Se observará máxima cautela y cumplimiento de las leyes de protección de datos de menores y personas con discapacidad, garantizando anonimato en datos y fotografías.

7. Estándares de Documentación

7.1. Documentación Textual en LaTeX a través de GitHub y Estructura Normalizada

Para la documentación textual y técnica, es decir, especificaciones, documentos de desarrollo, diseño y manuales que requieran redacción formal, utilizaremos **LaTeX** gestionado de forma centralizada y versionado a través de **GitHub**. Esta elección asegura un riguroso control de versiones mediante Git, homogeneidad tipográfica corporativa y generación determinista de documentos en formato PDF de alta calidad.

Para asegurar que todos los entregables compartan una imagen impecable y profesional, **la estructura que seguirá cada documento será obligatoriamente la siguiente:**

1. **Portada Institucional Normalizada:** Logotipo oficial de CLENCH Software Development, escudo corporativo de la Universidad de Granada, título formal del documento, subtítulo del proyecto AUTOVIDA, asignaturas (DGP/MDA), versión, fecha de emisión y listado completo de autores con roles asignados.
2. **Historial de Cambios y Control de Versiones:** Tabla de registro con versión, fecha, autores, resumen explicativo de cambios y estado de aprobación.
3. **Firma y Conformidad del Equipo:** Tabla de ratificación explícita por todos los miembros.
4. **Tabla de Contenidos (Índice):** Estructura paginada y jerarquizada generada automáticamente con numeración decimal.
5. **Introducción y Objetivos del Documento:** Propósito, justificación y alcance del documento.
6. **Cuerpo Central de Contenido:** Secciones y subsecciones numeradas de forma jerárquica, tablas con formato corporativo, figuras legibles y recuadros de aviso.
7. **Conclusiones y Sigüientes Pasos:** Síntesis analítica y acciones inmediatas.
8. **Referencias y Anexos:** Fuentes bibliográficas, enlaces a repositorios y glosarios.

Publicación, Versionado y Custodia: El código fuente .tex y todos los recursos asociados (figuras, imágenes y tablas) se mantendrán y versionarán en el repositorio Git de GitHub bajo la ruta docs/. Las modificaciones documentales se gestionarán mediante el flujo estándar de ramas y revisiones mediante *Pull Requests*, asegurando la aprobación del equipo. Los documentos finales se compilarán en formato PDF y se dispondrá adicionalmente de versiones en Markdown cuando sea conveniente para su consulta rápida en la propia plataforma de GitHub.

7.2. Documentación de Código: Doxygen

Con respecto a la documentación de código, hemos optado por usar **Doxygen**, aplicando así un estándar sencillo y fácil de entender para el grupo y cualquiera que pueda heredar el código, además de permitir la generación de documentación de código de manera dinámica a partir de los propios comentarios en los ficheros fuente.

7.3. Diseño y Modelado Software: UML con Visual Paradigm

Para la concreción del diseño de nuestra solución software (base de datos, definición de clases, diagramas de arquitectura y casos de uso) utilizaremos el estándar **UML**. Para ello nos apoyaremos en **Visual Paradigm**, por su capacidad de generar diagramas completos fácilmente y garantizar consistencia en la modelización de datos y componentes.

8. Estándares de Código

Utilizaremos la herramienta **Flutter** para la implementación. Se aplican las siguientes reglas de estilo y diseño:

8.1. Convenciones de Nomenclatura y Lenguaje

- **Idioma de Nomenclatura:** Los nombres de las variables, funciones, métodos, clases y archivos estarán escritos en ****inglés**** para mayor estandarización.
- **Variables y Funciones:** Se utilizará la forma lower-case / lowerCamelCase (ejemplo: `taskStatus`, `studentId`, `calculateTotalTime()`, `fetchActiveUsers()`).
- **Clases y Widgets:** Se empleará PascalCase de acuerdo con el estándar de Flutter (ejemplo: `TaskDetailCard`, `VisualTimerWidget`, `UserRepository`).
- **Constantes y Enumerados:** Se utilizará UPPER_SNAKE_CASE o lowerCamelCase según la convención oficial de Dart (`const int MAX_RETRY = 3;`).

8.2. Comentarios y Documentación en Código

Los comentarios de los scripts, funciones y demás estructuras se pondrán al inicio del documento o del respectivo bloque de código para mayor legibilidad y orden, empleando etiquetas y bloques compatibles con Doxygen (`///` o `/** ... */`). Describirán propósito, parámetros, retornos y excepciones controladas.

8.3. Principios de Calidad y Formato

- Código modular, desacoplado y siguiendo principios SOLID y Clean Code.
- Formateo automático mandatorio mediante `dart format` . antes de cada commit.

9. Plan de Gestión de Cambios

Ante cualquier cambio solicitado, se revisarán los requisitos afectados y se evaluará su impacto en la planificación, el diseño y el desarrollo de la aplicación. Una vez aprobado el cambio, se realizarán las modificaciones necesarias tanto en la aplicación como en la documentación y memoria del proyecto, manteniendo siempre actualizados los requisitos.

9.1. Procedimiento de Evaluación y Aprobación

1. **Registro de Solicitud:** Registro formal de la propuesta de modificación identificando origen y justificación.
2. **Análisis de Impacto:** Los coordinadores y el Gestor de Calidad evaluarán la repercusión sobre el cronograma en ClickUp, el esfuerzo en horas de los miembros, los costes y la accesibilidad.
3. **Decisión Formal:** Si el cambio es aprobado por el Coordinador, el Gestor de Calidad y el Moderador, se traslada al Product Backlog y se actualizan los diagramas UML y memorias.
4. **Preaviso de 3 Días:** Si la alteración afecta a las fechas de entrega pactadas en el pliego técnico, se comunicará formalmente al profesorado con un mínimo de **3 días de antelación**.

10. Control de Versiones (Método y Herramientas)

Utilizaremos la herramienta **GitHub** para el control de versiones, actualizando el repositorio compartido en cada sesión de prácticas, llevando a cabo un control supervisado y documentado correctamente, especificando qué cambios se han llevado a cabo.

10.1. Estrategia y Disciplina en el Repositorio

- **Actualización Sistemática:** Es obligatorio realizar commit y push de los avances generados al término de cada sesión presencial de prácticas, documentando de forma precisa el trabajo efectuado.
- **Control Supervisado y Pull Requests:** Las ramas funcionales de trabajo (feature/, bugfix/) deberán ser integradas mediante Pull Requests revisadas antes de su incorporación a la rama principal de desarrollo.
- **Prohibición de Código Roto:** Ningún integrante subirá código que no compile o rompa la batería de pruebas local (flutter test).

10.2. Convención de Commits (Conventional Commits)

Formato obligatorio: <tipo>: <descripción concisa>

- feat: Nueva funcionalidad implementada.
- fix: Corrección de un defecto o fallo.
- docs: Actualización exclusiva de documentación o actas.
- style: Cambios estéticos o de formateo sin impacto lógico.
- refactor: Modificación de código sin alteración funcional.
- test: Incorporación o modificación de pruebas unitarias/integración.
- a11y: Modificaciones específicas de accesibilidad (contrastes, VoiceOver/TalkBack).
- chore: Mantenimiento de configuración, dependencias o CI/CD.

11. Gestión de Calidad (Durante el Desarrollo y al Final, Herramientas)

Se revisará completamente todo el desarrollo al final de cada iteración y se corregirán los aspectos necesarios para alcanzar nuestros objetivos, asegurando una detección temprana de errores y validación continua del trabajo entregado.

11.1. Procedimiento de Calidad en Cada Iteración

Por cada iteración seguiremos rigurosamente los siguientes pasos:

1. **Definición de criterios de aceptación:** Discutiremos sobre lo que se ha de realizar en la iteración y qué criterios seguimos para determinar si se ha completado correctamente (Definition of Ready y Definition of Done).
2. **Revisiones de código:** Se revisará el código obligatoriamente en busca de errores antes de añadirlo a la rama de desarrollo de la nueva iteración.
3. **Análisis Estático Continuo:** Escaneo automatizado del código fuente en cada ciclo para identificar vulnerabilidades, duplicidades y fallos de estilo.
4. **Pruebas Unitarias y de Integración:** Ejecución de pruebas automatizadas sobre los nuevos módulos desarrollados en la iteración correspondiente.

5. **Pruebas de Regresión Iterativas (Iteraciones 2 y 3):** Al finalizar la Iteración 2 e Iteración 3, se verifica que los nuevos cambios no hayan roto funcionalidades previamente entregadas en la iteración anterior.
6. **Demo y Validación de Iteración:** Al cierre de cada iteración se realiza una revisión funcional con el cliente para validar el trabajo entregado.

11.2. Proceso de Hardening y Verificación Global

Al terminar la tercera iteración, se llevará a cabo el **proceso de hardening y verificación global**, el cual constará de:

- Una revisión funcional integral por parte del cliente.
- Baterías completas de pruebas de estrés, humo y regresión global.
- Una auditoría final exhaustiva que revisará las políticas de datos (RGPD) y analizará las vulnerabilidades del sistema.

11.3. Cuadro Oficial de Herramientas de Calidad

Fase / Ámbito	Herramienta	Aplicación en el Proyecto
Control de Versiones & CI/CD	GitHub	Gestión del repositorio, control de Pull Requests y ramas.
Análisis Estático & Estilo	SonarCloud / ESLint	Detección automática de bugs, vulnerabilidades y formato de código.
Pruebas Unitarias & API	Jest	Automatización de pruebas de lógica e integración de servicios.
Accesibilidad & UX	Lighthouse / WAVE	Evaluación de niveles de accesibilidad (contraste, etiquetas, lectura).
Gestión de Tareas y Bugs	Jira	Trazabilidad de requisitos, incidencias y feedback de las demos.

12. Plan de Medición del Desempeño, Recompensas y Castigos

12.1. Registro y Control del Esfuerzo Individual

Cada miembro cumplimentará semanalmente su hoja personal de registro de dedicación horaria en la plantilla oficial (docs/plantillas/plantilla_registro_horas.md). El balance consolidado será auditado por el Coordinador y el Gestor de Calidad en cada iteración para garantizar un reparto equilibrado de la carga.

12.2. Sistema de Recompensas y Reconocimiento

- **Reconocimiento Público Interno:** En el acta de retrospectiva de cada iteración se hará mención explícita al miembro o rol destacado por su proactividad, aportes de calidad o apoyo extraordinario a sus compañeros.
- **Prioridad de Elección Técnica:** Aquellos integrantes con desempeño excelente y cumplimiento anticipado de entregables tendrán preferencia al seleccionar sus componentes de trabajo en el sprint sucesivo.

- **Objetivo Común de Calificación Máxima:** Se busca promover la excelencia compartida para que todos los integrantes obtengan la máxima calificación académica (10) en la evaluación de prácticas.

12.3. Régimen Sancionador Interno (Castigos y Penalizaciones)

Para asegurar la responsabilidad individual y salvaguardar los resultados del grupo frente a actitudes negligentes:

1. **Inasistencia a Reuniones:** La falta injustificada a una reunión conllevará apercibimiento interno. La acumulación de **dos faltas injustificadas** a reuniones oficiales acarreará amonestación formal en acta y se traducirá en penalización en la nota individual remitida a los profesores responsables.
2. **Incumplimiento de Entregables o Plazos:** El retraso o no entrega de tareas comprometidas sin previo aviso justificado de al menos **48 horas** motivará la reasignación de urgencia de la tarea y figurará con calificación desfavorable en el informe individual del implicado.
3. **Reincidencia en Falta de Calidad:** Subir código defectuoso que impida el trabajo del resto obligará a su corrección inmediata fuera del cómputo computable de horas.

Documento consensuado y aprobado por el Equipo CLENCH Software Development.
Grado en Ingeniería Informática — Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación.
Universidad de Granada, Curso Académico 2026–2027.